3

Concurrency En HPC computing

© 2008 Pearson Education, Inc. All rights reserved.

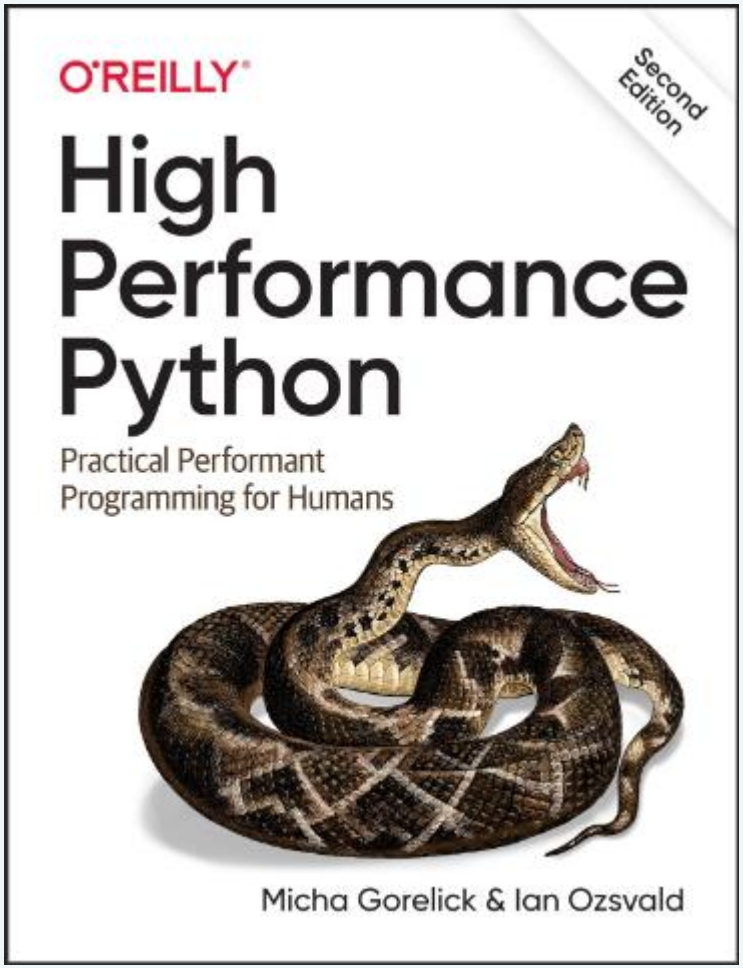
Inhoud

1. Inleiding
2. Programmeren onder Linux
3. Concurrency & HPC computing
   3.1 High performance computing (HPC)
   3.2 Code profiling
   3.4 HPC concepten
   3.5 Synchronous vs asynchronous
   3.6 Concurrency vs parallelism
   3.7 Multiprocessing vs Multithreading
   3.8 Asyncio
   3.9 Multithreading
   3.10 Multiprocessing

© 2008 Pearson Education, Inc. All rights reserved.

Referenties

- HPC Python Boeken:



+ tal van online referenties verder in de slides

© 2008 Pearson Education, Inc. All rights reserved.

3.1 High Performance Computing

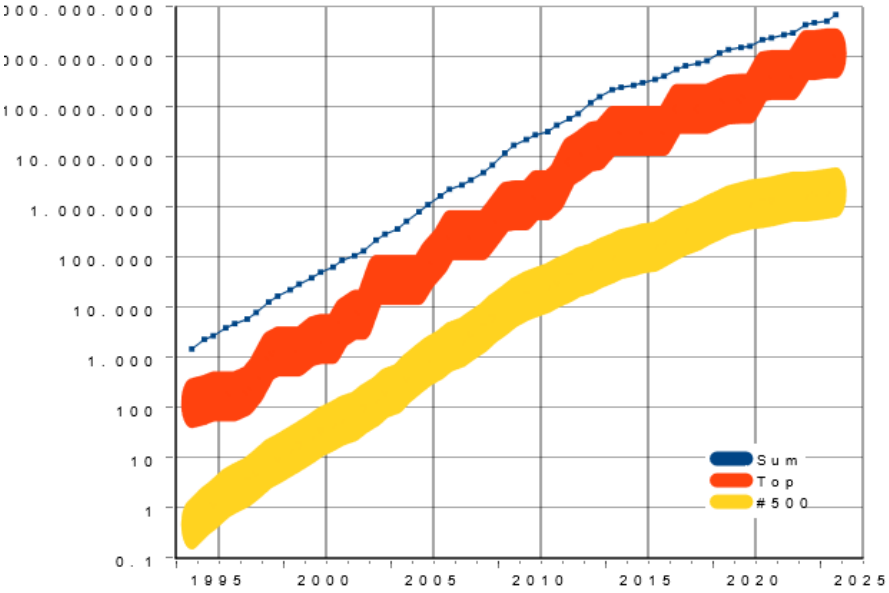
- Het aggregeren van computing resources ter verhoging van de computationele performantie in vergelijking met een desktop system
- Voor het oplossen van grote computationele problemen in wetenschap, engineering en business
- Historische evolutie: supercomputers → HPC clusters
  - Van gespecialiseerde (dure) hardware naar 'clusters' of 'grids' van wijd verkrijgbare commerciële systemen

Cray supercomputer < 1990

© 2008 Pearson Education, Inc. All rights reserved.

3.1 High Performance Computing

- **TOP500 project: maakt ranking van 500 meest performante non-distributed computer systemen**
- **Werkt met Benchmarks: bvb. LINPACK Benchmarks voor oplossen van nxn system lineaire vergelijkingen (dense matric algebra), of Linpack's Highly Parallel Computing benchmark, HPLinpack**
- **FLOPS : aantal 64-bit floating-point operations, zoals bvb. optellingen en vermenigvuldigingen and multiplications die een computer kan uitvoeren per seconde**
- **Huidige koploper: Frontier (Oak Ridge) met 1102 petaFlops**
- **Naast non-distributed systems ook distributed clusters: cloud computing**



Performantie vs tijd in GFLOPS

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Code Quality

- High quality code is code die:
  - Precies doet wat ze moet doen: vastgelegd in requirements
  - Ze bevat geen defecten of problemen (bugs): ze moet breed toepasbaar zijn, dus onderzoek randgevallen, gevallen waar code vastloopt of ongewenst gedrag vertoont
  - Ze is makkelijk te lezen, te onderhouden en uit te breiden: bevordert teamwork, herbruikbaarheid en toevoeging van nieuwe functionaliteiten:
    - Coding style guides geven een richtlijn
- Python Enhanced Proposal #8 (PEP-8) bevat alle details/afspraken over degelijk coderen in Python
  - Zie: https://peps.python.org/pep-0008/
- Tools die Python code checken op syntax en stijl (nb. De PEP8 standard) en gevaarlijke logica identificeren (logical lint):
  - Pylint
  - PyFlakes
  - Pycodestyle
  - pydocstyle

Installeer ze via pip install
Daarna gebruik in PyCharm:
python - How to run Pylint with PyCharm - Stack Overflow
Of in je terminal via:> pylint your_code.py

Referentie: Python Code Quality: Tools & Best Practices – Real Python

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Code Performance

- Code performance:
  - Zal bepalen of je code langzaam of snel wordt uitgevoerd (execution time)
  - Wordt bepaald door verschillende factoren:
    - Efficient gebruik van het geheugen
    - Efficient gebruik van de CPU
    - Efficient gebruik van file IO en network IO
- Code refactoring:
  - Aanpassen van code in termen van performance zonder dat de functionaliteit verandert
  - Gebeurt iteratief via kleine aanpassingen en de juiste tests: testen op correcte functionaliteit en op performance
  - Kan ook gebeuren in functie van leesbaarheid, beknoptheid, herbruikbaarheid

Referentie: Code Refactoring Best Practices – with Python Examples (freecodecamp.org)

© 2008 Pearson Education, Inc. All rights reserved.

3.2 Code Performance

- Dominerende factoren in code performance:
  - Memory management:
    - Python gebruikt dynamic memory allocation: objecten worden op runtime niveau gecreerd en verwijderd (door de garbage collector)
    - De garbage collector verwijdert een object als de reference counter op nul staat
    - Je kan de garbage collector helpen door een “del” statement uit te voeren
  - CPU management:
    - Algoritmes hebben een algorithm complexity, uitgedrukt in big O notation: O(n), met n de unit size of aantal bits om de input te representeren. Gebruik bij voorkeur de lage n-orde algoritmes of gebruik kleine n
    - Loop optimization: Loops gebruiken veel CPU, check of loops niet anders gestructureeerd kunnen worden: via list comprehensions, generators, filters etc…
    - Voor numerieke datastructuren: gebruik zoveel mogelijk gevectoriseerde operaties op data: Zie NumPy vectorization
  - I/O operaties:
    - Lezen van of schrijven naar files of network links is tijdrovend
    - Gebruik bufferening: zie documentatie in de Python open() functie
    - Gebruik asynchronous I/O via de asyncio module in Python

Referentie: Python Execution Time: Factors Behind Slower Execution - Coding Apex

© 2008 Pearson Education, Inc. All rights reserved.

3.3 Code Profiling

- Tools om code performantie te meten: Profilers
  - Memory profiler:
    - Te installeren via pip install memory-profiler
    - Run je code via:
      - mprof run <your_pythoncode.py>, of
        mprof run –include-children <your_pythoncode.py> voor multiprocessing
      - mprof plot
    - Of werk in je code met @profile decorator bij elke functie die je wil profilen
      - Import memory_profiler
      - Voorbeeld: Zie word_extractor.py en profiler_test.py

Referentie: Memory profiling in Python using memory_profiler - GeeksforGeeks

© 2008 Pearson Education, Inc. All rights reserved.

3.3 Code Profiling

- Tools om code performantie te meten: Profilers
  - Run time performantie:
    - Python voorziet standard 2 profilers: profile en cProfile
    - Profilers vergen zelf tijd
    - Profile is trager dan cProfile maar is in staat om threads te verwerken
    - Run in de terminal via:
      - python -m cProfile -o profile_results.prof your_script.py
      - Analyseer met:
        - python -m pstats profile_results.prof
        - Dit opent een interactive sessie waar je pstats functies kunt aanroepen: strip, sort, stats
    - Of importer in je code the cProfile en pstats library, en roep de Profile en Stats methods aan:
      - Profile methods: run(), runcall()
      - Stats methods: strip_dirs.(), sort_stats(), print.stats()

Referentie: The Python Profilers — Python 3.12.2 documentation
Python 102: How to Profile Your Code - Mouse Vs Python (pythonlibrary.org)
Handy Python profiling analysis with pstats' interactive browser - Stefaan Lippens inserts content here

© 2008 Pearson Education, Inc. All rights reserved.

3.4 HPC computing concepten

- **HPC infrastructuur leent bij uitstek tot parallelisatie en/of concurrency dmv haar multiprocessing architectuur**
- **Multiprocessing: Gebruik van 2 of meer CPU's op een computationeel system**
- **Distributed memory: een multiprocessor systeem waarbij elke CPU geassocieerd is met een eigen geheugen. Berekeningen gebeuren enkel op 'lokale' data.**
- **Shared memory: Dezelfde geheugenruimte wordt gedeeld door verschillende CPU's**
- **Parallellisatie: Het simultaan uitvoeren van verschillende berekeningen: executie van elk process is gelijktijdig**
- **Concurrency: De levensduur van verschillende processen overlapt in de tijd: executie hoeft niet noodzakelijk parallel te verlopen, maar bvb. via time-sharing**
- **Thread: Kleinste reeks programma instructies die onafhankelijk kan behandeld worden door het operating system (scheduler), en is onderdeel van een lopend uitvoerbaar programma (process)**
- **Multithreading: gebruik van uitvoerende programa dat meerdere threads bevat. Threads delen de resources van het process, maar kunnen onafhankelijk worden uitgevoerd.**

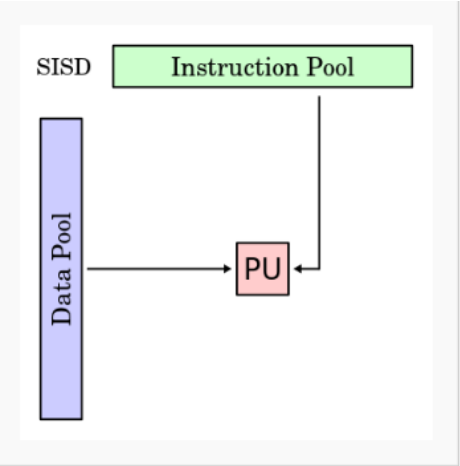
Flynn's taxonomy (multiprogramming context)

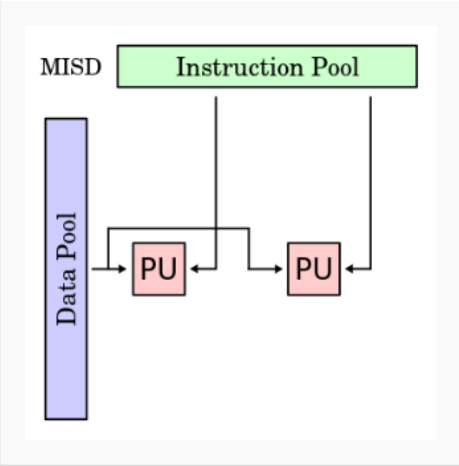
	Single instruction stream	Multiple instruction streams	Single program	Multiple programs
Single data stream	SISD	MISD		
Multiple data streams	SIMD	MIMD	SPMD	MPMD

© 2008 Pearson Education, Inc. All rights reserved.

3.4 Flynn Taxonomie

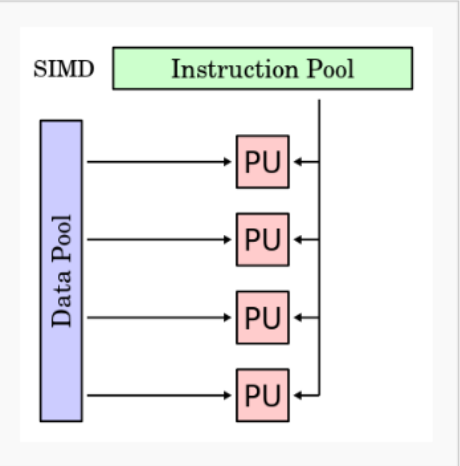
•Uniprocessor, unicore

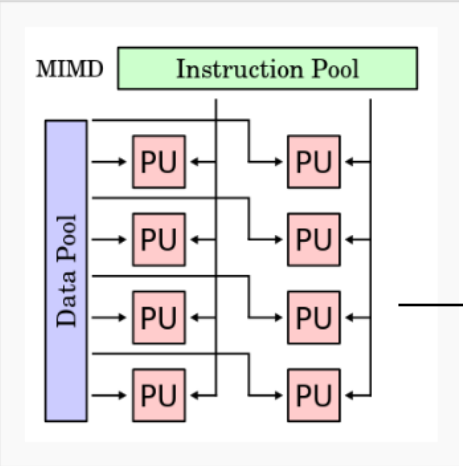




•Pipeline architecture

• GPU, …





• Multicore, distributed systems

•Meest gebruikelijke HPC model

Verfijning van de MIMD

- SPMD:Meerdere autonome processoren die simultaan hetzelfde programma uitvoeren, maar eventueel op andere instructie-posities
- MPMD: Meerdere autonome processoren die minstens 2 onafhankelijke programma's uitvoeren.

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Synchronous vs asynchronous

- Thread: series of instructions (a block of code) that exist as a unit of work
  - Een operating system kan enkel 1 thread uitvoeren op 1 CPU core
  - Een operating system kan wel de illusie geven dat het meerdere treads uitvoert door elke thread gedurende een beperkte tijd uit te voeren, deze dan te pauzeren en te switchen naar een andere thread
- Synchronous processing:
  - Betekent dat alle taken verbonden zijn of co-dependent, ze zijn afhankelijk van elkaar: bvb taak B moet wachten tot taak A is uitgevoerd
  - De start en stoptijden van elke taak zullen nooit overlappen
  - Kan ofwel single threaded of multi-threaded zijn

Synchronous (one thread):

```
1 thread ->   |<---A---->||<----B---------->||<------C----->|
```

Synchronous (multi-threaded):

```
thread A -> |<---A---->|
                       \
thread B ------------>   ->|<----B---------->|
                                             \
thread C ---------------------------------->   ->|<------C----->|
```

[Referentie: Asynchronous vs synchronous execution. What is the difference? - Stack Overflow]

© 2008 Pearson Education, Inc. All rights reserved.

3.5 Synchronous vs asynchronous

- Asynchronous processing:
  - Betekent dat alle taken onafhankelijk zijn: ze zullen zowel in hun initiatie (wanneer ze starten) als in hun executie (hun uitvoering hangt niet af van resultaten uit een andere taak)
  - De start en stoptijden van asynchrone taken zullen overlappen
  - Kan ofwel single threaded of multi-threaded zijn

Asynchronous (one thread):

```
          A-Start ------------------------------------------ A-End
            | B-Start ----------------------------------------|--- B-End
            |    |      C-Start ------------------- C-End      |      |
            |    |       |                           |         |      |
            V    V       V                           V         V      V
 1 thread->|<-A-|<--B---|<-C-|-A-|-C-|--A--|-B-|--C-->|---A---->|--B-->|
```

Asynchronous (multi-Threaded):

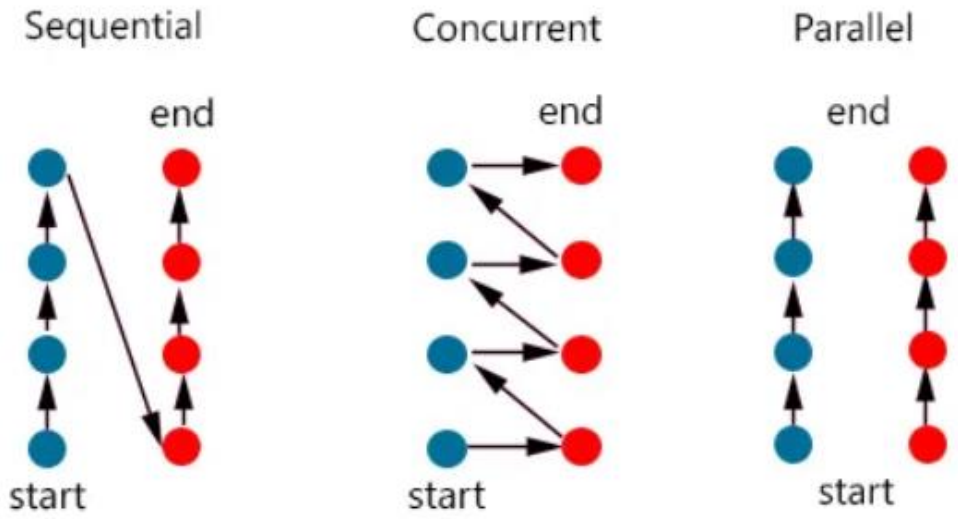
```
thread A ->     |<---A---->|
thread B ----->     |<----B---------->|
thread C --------->     |<------C--------->|
```

  - Synchronous of asynchronous executie hebben dus in principe niets te maken met threads: MAAR synchronous programs zullen heel vaak een enkele thread hebben en soms zullen meerdere threads synchroon moeten worden uitgevoerd

[Referentie: Asynchronous vs synchronous execution. What is the difference? - Stack Overflow]

© 2008 Pearson Education, Inc. All rights reserved.

3.6 Concurrency vs. parallelism

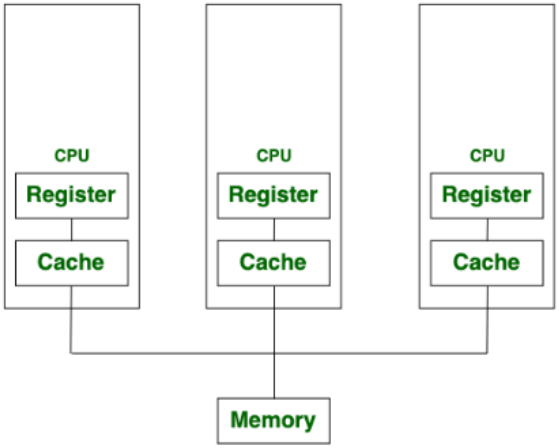


- Concurrent processing:
  - Verschillende taken kunnen starten, uitvoeren en stoppen in overlappende tijdsperiodes, maar ze hoeven niet gelijktijdig uitgevoerd te worden: ze worden typisch uitgevoerd op een enkele CPU core (maar hoeft niet per se)
- Parallel processing:
  - Wanneer meerdere threads simultaan uitgevoerd worden en gelijktijdig voortgang maken: vereist altijd meerdere CPU cores
- Concurrent processing is eigenlijk een meer algemene vorm van parallelisatie: ook wel virtuele parallelisatie genoemd

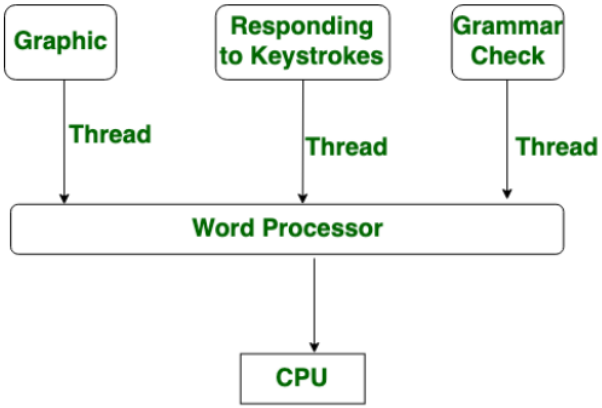
Referentie: language agnostic - What is the difference between concurrency and parallelism? - Stack Overflow

© 2008 Pearson Education, Inc. All rights reserved.

3.7 multiprocessing vs multithreading



Multiprocessing



Multithreading

- Multiprocessing: een vorm van parallelisatie die een specifieke vorm van concurrency is
  - Vereist altijd meerdere CPU cores
  - Elk process wordt uitgevoerd eigen core die eigen geheugen register en cache heeft
  - Alle processen worden simultaan uitgevoerd
  - Beter geschikt voor CPU gelimiteerde taken (CPU-bound tasks)
- Multithreading:
  - Een process kan meerdere threads creeren die simultaan kunnen worden uitgevoerd, ook op meerdere CPU cores indien nodig
  - MAAR: threads creeren is economischer dan processen en threads delen dezelfde address space (geheugen blok)
  - Beter geschikt voor I/O gelimiteerde taken (I/O bound tasks)

Referentie: Difference between Multiprocessing and Multithreading - GeeksforGeeks

© 2008 Pearson Education, Inc. All rights reserved.

3.8 Asyncio

- Asynchronous IO:
  - In Python aangeboden via de asyncio library
  - pip install --upgrade pip aiohttp aiofiles
  - Coroutine:
    - een gespecialiseerde functie die asynchroon kan worden uitgevoerd
    - Wordt gedefinieerd met async def naam_coroutine(argumenten)
  - Keyword await: passes control to the event loop, ot suspends the execution of the surrounding coroutine: maw. “wacht tot de executie van de aangeroepen coroutine klaar is
  - Awaitable object:
    - Een aandere coroutine gedefinieerd met async
    - Een object dat de __await__ method heeft geimplementeerd (die een iterator moet returnen)
  - asyncio.gather():
    - Laat toe om meerdere awaits te groeperen
    - Eens gegroepeerd zullen de awaitables hun taken asynchroon uitvoeren, op hun resultaten gewacht worden waar nodig
  - asyncio.Queue():
    - Laat toe om data te delen onder coroutines
    - De data producers zetten hun data in de queue
    - De data consumers halen data van de queue
    - Producers en consumers zijn onafahankelijk van elkaar, ze weten zelfs niet hoeveel van elk er zijn
  - Voorbeelden: zie asynchiotest1.py, asynchiotest2.py, asynchiochain.py, asincio_queue.py

Referentie: Async IO in Python: A Complete Walkthrough – Real Python

© 2008 Pearson Education, Inc. All rights reserved.

3.9 Multithreading

- Python Threads:
  - Hebben een eigen execution flow
  - Worden bescheven door een Tread Control Block (TCB)
  - Ze kunnen dus verschillende dingen tegelijk doen
  - MAAR: In Standard Python zullen ze niet noodzakelijk gelijktijdig (parallel) worden uitgevoerd
  - Dit komt door de Python GIL (Global Interpreter Lock) die enkel toelaat dat 1 thread gebruik maakt van de python interpreter
  - De GIL vermijd race condities in Python: conditie waarin verschillende treads gelijktijdig dezelfde data gebruiken
  - In Python 3 laat de GIL toe dat threads die tijdelijk geen CPU gebruiken hun gebruik van de GIL los te laten
  - DUS: Python threads kunnen een programma versnellen dat typisch IO bound is

Referentie: An Intro to Threading in Python – Real Python
What Is the Python Global Interpreter Lock (GIL)? – Real Python

© 2008 Pearson Education, Inc. All rights reserved.

3.9 Multithreading

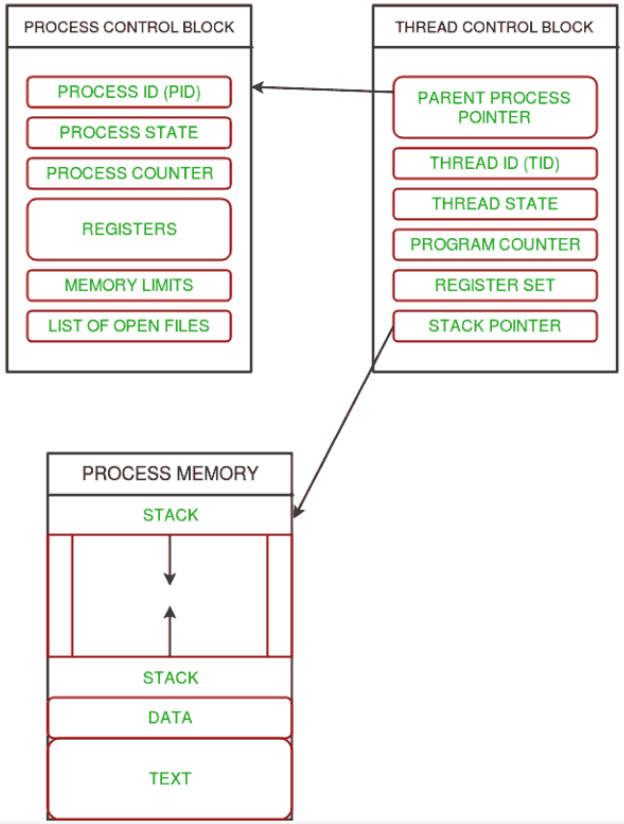
- Python Process:
  - Is een instance van een python programma dat wordt uitgevoerd
  - Bevat:
    - Een uit te voeren programma
    - De data geassocieerd met dat programma (variables, workspace, buffers, …)
    - De context van uitvoering (Process state): Is het programma actief of niet?
- Thread Control Block (TCB):
  - Bevat:
    - Thread Identifier (TID): unike identifier toegewezen aan elke nieuwe thread
    - Stack pointer: Wijst naar de stack van elke thread die variabelen lokaal aan de thread bevat
    - Program counter: Een register dat de adressen van alle instructies uit de thread bevat
    - Thread state: running, ready, waiting, starting of done
    - Thread register set: register gerserveerd voor alle berekeningen binnen een thread
    - Parent Process Pointer: pointer naar de process control block van het process waarbinnen een thread gecreerd is

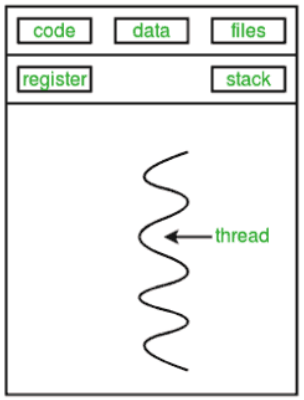
Referentie: Multithreading in Python - GeeksforGeeks

© 2008 Pearson Education, Inc. All rights reserved.

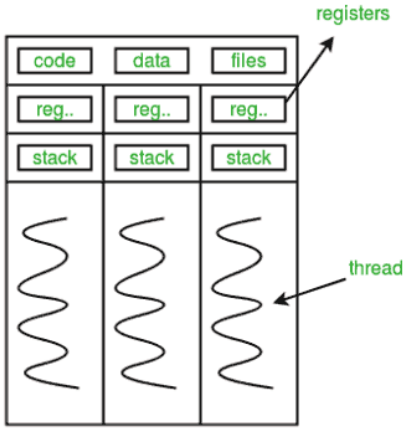
3.9 Multithreading

- Elke thread heeft zijn eigen register set en local variables (opgeslagen in de stack).
- Alle threads van een process delen global variables (opgeslagen in de heap), en de program code.
- Multithreading is gedefinieerd als de mogelijkehid van een processor om meerdere threads concurrent uit te voeren. In een simple, single-core CPU, wordt dit bereikt door frequent te switchen tussen threads.

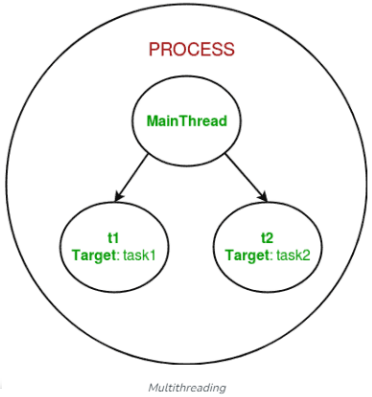


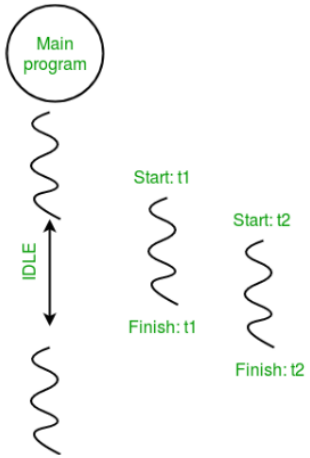


single-threaded process



multithreaded process





Referentie: Multithreading in Python - GeeksforGeeks

Inc. All rights reserved.

3.9 Multithreading

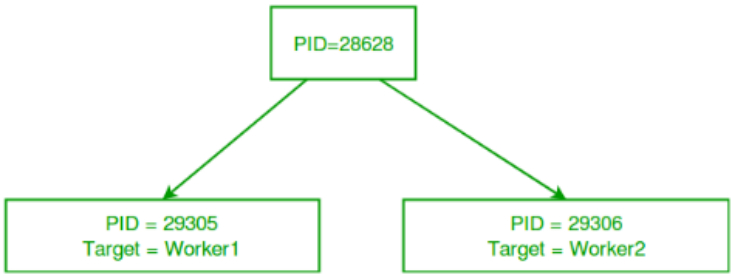
- Multithreaded python code:
  - Te gebruiken door de threading library te importeren
  - Bij creatie van een Thread object geef je het als argumenten de naam van een functie en een lijst van parameters mee
  - Je kan uiteraard ook Object methods als threads uitvoeren, en je kan ook een object laten erven van de threading.Thread klasse
  - Indien een programma treads uitvoert die geen deamon threads zijn, dan wordt het programma pas gestopt als alle threads klaar zijn
  - Indien je een thread als deamon thread definieert, dan wacht het programma niet totdat deze klaar is
  - Wil je toch expliciet wachten tot een thread klaar is, dan moet je de join() method gebruiken
  - Meerdere Threads kunnen (quasi) gelijktijdig starten, maar zullen typisch niet in de zelfde volgorde klaar zijn, zelfs niet als het threads met dezelfde functie zijn: dit is operating system afhankelijk
  - Door de aanpassing van de GIL in Python3 zijn race condities weer mogelijk!
  - Voorbeelden: zie Multithreading_ex1.py, Multithreading_ex2.py, Multithreading_lock.py, Multithreading_deamon.py

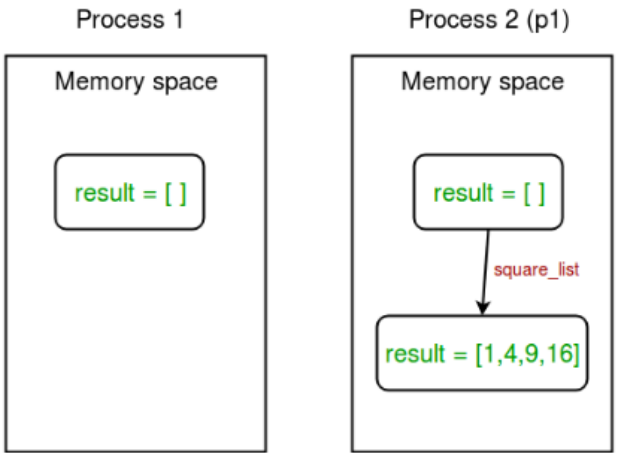
Referentie: An Intro to Threading in Python – Real Python
What Is the Python Global Interpreter Lock (GIL)? – Real Python

© 2008 Pearson Education, Inc. All rights reserved.

3.10 Multiprocessing

- Multiprocessing refereert naar de mogelijkheid van een programa om meerdere CPU's of CPU cores te gebruiken
- Om een Process te maken creeer je een object van klassen Process uit de python multiprocessing library
- Elk process wordt onafhankelijk uitgevoerd en heeft zijn eigen geheugen ruimte
- Een Process object krijgt bij creatie minstens twee argumenten: De naam van een functie die in dat process moet worden uitgevoerd en haar argumentenlijst
- Een process wordt gestart via de methos process.start(): nadat dit gebeurt kan het programma dat een process gestart heeft ook zelf weer verder werken
- Indien je wil dat het programma dat een process gestart is niet wordt beeindigt terwijl processen nog lopen moet je de process.join() method aanroepen voor elk van de gestarte processen
- Je kan vele processen genereren en beheren via het multiprocessing.Pool object
- Bescherm globale data met het Lock object





Sharing data between processes

Referentie: Multiprocessing in Python | Set 1 (Introduction) - GeeksforGeeks

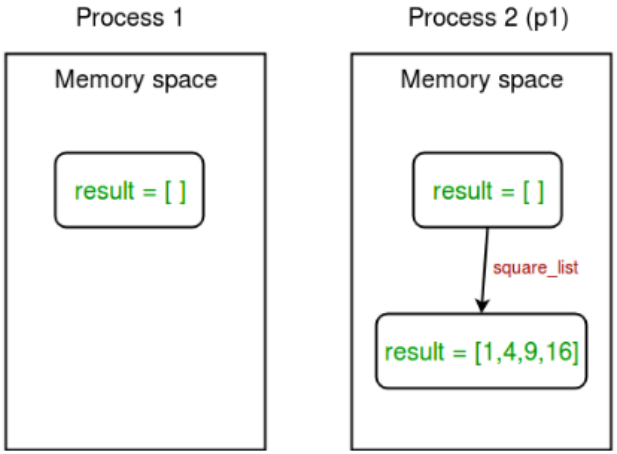
© 2008 Pearson Education, Inc. All rights reserved.

3.10 Multiprocessing

- Om data te delen tussen verschillende processen moet men met shared memory werken
- Dit kan opverschillende manieren:

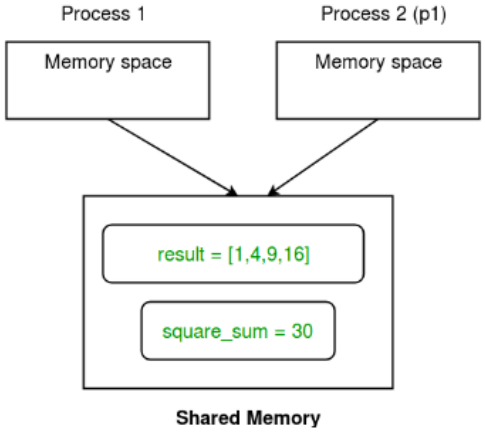
1. Via de multiprocessing.Value() (voor de waarde van 1 enkel object of variabele) en multiprocessing.Array() (voor een lijstje van values)
2. Via de manager van het automatisch gecreerde server proces bij de start van elk python programma

Geen gedeelde data

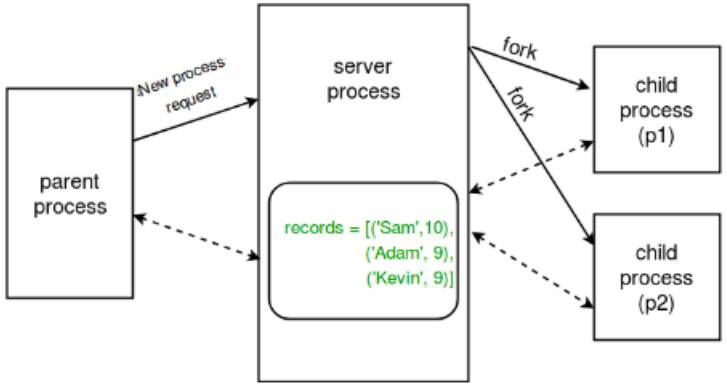


Sharing data between processes

Gedeelde data via Array() en Value()



Gedeelde data via een Manager die een Server Process beheert



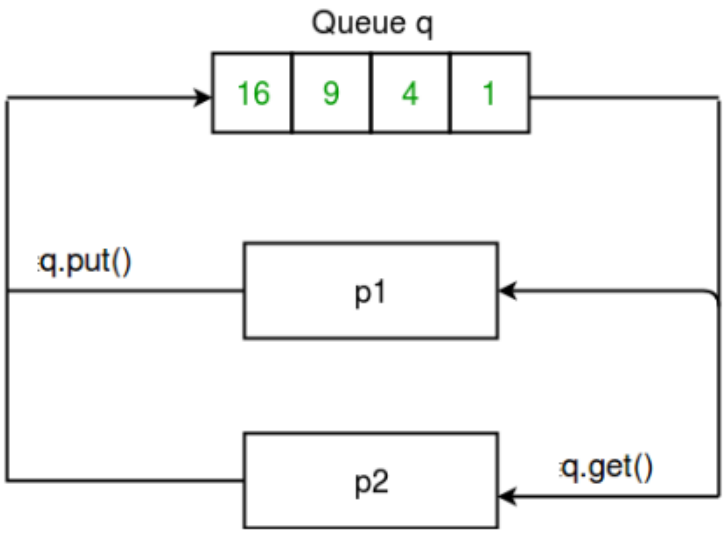
Communication between processes

© 2008 Pearson Education, Inc. All rights reserved.

3.10 Multiprocessing

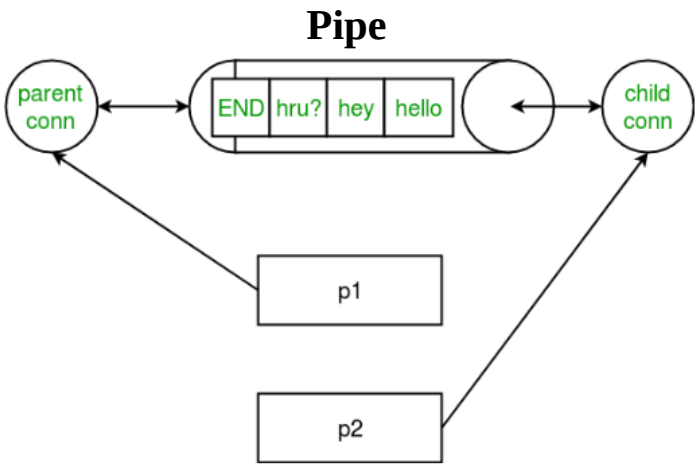
- In je met heel veel processen werkt of een 1-1 communicatie wil opzetten tussen 2 specifieke processen, dan zijn er efficientere manieren
- Dit kan opverschillende manieren:
  - Queue

1. Via een Queue object: elk process kan toegang krijgen tot de queue
2. Elk process kan via de Queue.put() data naar de queue sturen en via de Queue.get() data uit de queue halen
3. Een queue is een FIFO structuur
4. Queues zijn thread en process safe: geen data corruptie mogelijk (maar kost meer resources)



  - Pipe

1. Via een Pipe object dat enkel twee eindpunten heeft
2. Te verkiezen als je enkel tweerichtingsverkeer tussen 2 processes wil
3. Elk process kan via de Pipe.send() data naar de pipe sturen en via de Pipe.recv() data uit de pipe halen
4. Data in de pipe kan corrupted worden indien meer dan 1 process tegelijk data naar de pipe stuurt



© 2008 Pearson Education, Inc. All rights reserved.

3.10 Multiprocessing

- Er staan een heleboel voorbeelden van Multiprocessing python code op Blackboard:
  - Multiprocessing_ex1.py, Multiprocessing_ex2.py, Multiprocessing_ex3.py, Multiprocessing_ex3_lock.py, Multiprocessing_pool.py, Multiprocessing_server.py, Multiprocessing_pipe.py, Multiprocessing_queue.py

Referenties:

Multiprocessing in Python | Set 1 (Introduction) – GeeksforGeeks

Python Multiprocessing Archives - Super Fast Python

© 2008 Pearson Education, Inc. All rights reserved.

3.11 Conclusie

- **Bestudeer je code quality met Linters**
- **Bekijk dan je code en kijk of er mogelijkheden tot refactorisatie zijn**
- **Bestudeer je algoritmes om de CPU complexiteit te bepalen**
- **Profileer je code ifv. Geheugen en CPU gebruik**
- **Pas nadat je begrijpt waar de bottlenecks zitten kan je overgaan tot concurrency**

© 2008 Pearson Education, Inc. All rights reserved.